

E Appendix Discussion

E.1 Planned End-to-End test

To further ensure the integrity of the SpisNemt system, a three-step End-to-End test was planned. The purpose of this was to validate the data transport from user input to ML predictions. Although a formal End-to-End test with external participants was not prioritized to make room for the manual Acceptance test, the design of said test with external participants serves as proof of the system’s logical consistency:

Phase 1: Onboarding and Configuration

Participants are tasked with registering a new account and logging in. After successfully logging in, the participant navigates to the account page and sets their specific preferences with area and category.

Phase 2: Ingredient searching

Participants are instructed to utilize the native camera for ingredient scanning and the manual text input to search through recipes from the third party API.

Phase 3: Recipe discovery

Participants are required to submit the search query, and verify that the returned recipes match the preferences they put into their user profile. The match is graded with the "Perfect Match", "Good Match" and "Poor Match" badges as defined in figure 23 and 24 of section 11.3.3.

E.2 Testing strategy reflection

While the final acceptance test given in section 12 confirms that system fulfills the functional requirements of section 6.1, the development process faced some trade-offs in regards to modular verification. Due to at strict deadline of the project and the complexity of the technical stack, especially the client side machine learning, system validation was prioritized over unit verification.

This results in technical debt, which primarily stems from the frontend and machine learning modules. Managing the Expo builds took more time and effort than expected, so automated testing was prioritized less in favor of stabilizing the pipeline between the client-side interface and the Firebase backend.

To mitigate the risks that come with a lack of modular testing, a manual test as defined in section 12 was executed. This provides evidence of system integrity and satisfies the non-functional reliability requirements. While the investigation of EAS testing for creating automatic user flow testing is highly relevant for future supportability, the implementation of this is not critical for the delivery of a functional prototype of the SpisNemt system.

E.3 Expo difficulties

While the use of Expo on paper is a solid choice for the frontend it does not come without difficulties. As mentioned in section 8.1, a reason for choosing Expo is the sandbox app,

Expo Go, which speeds up the process of compiling your code and viewing the progress in an app format. However, the Expo Go app is only usable as long as the libraries used are part of the Expo SDK. This causes problems as the chosen authentication technology, discussed in section 8.2 and section E.5, is a third party library from Google.

Although another authentication implementation is eventually opted for, the Expo Go problems arise again when implementing the machine learning models into SpisNemt. The application requires loading a heavy InceptionV3 architecture consisting of a JSON topology and 21 binary weight shards. While Expo Go supports WebGL hardware acceleration through *expo-gl*, its sandboxed environment introduces strict memory limits and utilizes pre-compiled bindings. This means that when the *@tensorflow/tfjs-react-native* library attempts to read the binary assets via *bundleResourceIO* and pass them through the JavaScript bridge to the GPU, Expo Go experiences bottlenecks, memory exhaustion and silent crashes.

To bypass these limitations, development builds are required using *npx expo run:android*. This compiles a dedicated native application, allowing the underlying JavaScript engine and WebGL context to use the device's hardware and memory, which is necessary for executing complex convolutional neural networks on the client-side.

E.4 Database reflection

As stated in section 8.3, Firebase JS SDK was chosen as the database for this project, because it is compatible with Expo Go and offline persistence is possible. As offline functionality was never implemented, not much research of how to implement was done. However, after the development phase was finished, some research was done of how to implement offline persistence, where it was discovered that Firebase JS SDK does not support offline persistence for Expo in android and iOS environments, but only web. Instead of Firebase JS SDK, the other Firebase option, React Native Firebase, should have been chosen, as it supports offline persistence for android and iOS environments in Expo out of the box. As stated in section 8.3, React Native Firebase is not supported by Expo Go, forcing development builds to be used instead, causing significant build times, and requiring an android emulator to run.

E.5 Authentication difficulties

During development it is revealed that by using *@react-native-google-signin/google-signin* which is a third party library, Expo Go can't be used because Expo Go only renders installed packages, which only includes React native and Expo libraries. By using *@react-native-google-signin/google-signin* development builds are required in order to run the app. These builds can only be run on an emulator or an actual phone, by using the built .apk file, transferred by cable when building. This process is more time consuming than using Expo Go and will slow down development time as well as make it more tedious. As a result, *AuthSession* is opted for instead of *@react-native-google-signin/google-signin*.

The swap from *@react-native-google-signin/google-signin* to *AuthSession* as authentication for the login system reveals issues when implementing Google as an OAuth sign in option. The reason for *AuthSession* not working is that Google has increased the security of OAuth login for mobile apps in recent years. After additional research regarding authentication in Expo, it is stated in an authentication guide that *AuthSession* can't be used in Expo Go.

This key information is missed during initial research on *AuthSession*. To use *AuthSession*, development builds must be used which significantly slows down development time, as the builds take longer to build. The switch to *AuthSession* is revealed to be a poor decision.

Instead of using *AuthSession* or *@react-native-google-signin/google-signin* to make google login possible, a custom login solution with firebase authentication is opted for through the *firebase/auth* library. This removes google as a signin option, which was initially thought to be the easiest way to implement a login system. Using *firebase/auth* turns out to be simpler to implement than initially thought, as the *firebase/auth* library provides functions for creating accounts, sign in, and sign out called *createUserWithEmailAndPassword*, *signInWithEmailAndPassword*, and *signOut* respectively. *Firebase/auth* is supported in Expo Go as it doesn't require native modules like *@react-native-google-signin/google-signin* does. In addition Cloud Firestore and firebase authentication collaborates quite well since they are of the same ecosystem. With *firebase/auth*, an extra layer of first authenticating with Google, then exchange that token into Firebase authentication credentials isn't needed anymore.